UNIDAD DIDÁCTICA 5

BASES DE DATOS XML

MÓDULO PROFESIONAL:
ACCESO A DATOS

Índice

RESUMEN INTRODUCTORIO 2
INTRODUCCIÓN 2
CASO INTRODUCTORIO 3
1. BASES DE DATOS NATIVAS XML 4
1.1 Comparativa con bases de datos relacionales 6
1.2 Principales gestores comerciales y libres 9
1.3 Estrategias de almacenamiento 11
1.4 Colecciones y documentos 14
1.5 Ventajas y Desventajas de las Bases de Datos Nativas XML 14
2. API JAVA ESTÁNDARES PARA BD-XML NATIVAS 17
2.1 API XQJ 17
2.2 Establecimiento y cierre de conexiones 19
2.3 Realización de consultas. Clases y métodos 21
3. USO Y CREACIÓN DE BASES DE DATOS XML CON BASEX 24
3.1 Interfaz y creación de una base de datos 25
3.2 Añadir, modificar y eliminar documentos 26
3.3 Creación y borrado de colecciones 28
3.4 Lenguaje de consulta para XML: XQuery (Xml Query Language) 30
3.4.1 Instrucciones basadas en Xquery 31
3.4.2 Instrucciones basadas en XPath 33
3.5 Clases y métodos Java para usar con BaseX 34
3.5.1 Incluir BaseX dentro de nuestros proyectos 35
3.5.2 Ejecutar comandos de forma local. Añadir, modificar o eliminar documentos y colecciones 36
3.5.3 Tratamiento de las excepciones 42
RESUMEN FINAL 46

RESUMEN INTRODUCTORIO

En esta unidad se introducirán los conceptos necesarios para comenzar a trabajar con bases de datos de tipo XML.

En el primer apartado veremos cómo se definen las bases de datos XML, qué diferencias y semejanzas tienen con las bases de datos relacionales, así como los diferentes tipos que nos encontramos dentro las bases de datos XML para finalizar mostrando ejemplos de bases de datos nativas XML actuales.

En segundo lugar, mostraremos cómo incorporar las librerías que Java nos proporciona para poder trabajar con el lenguaje XQuery dentro de nuestros proyectos Java y ficheros XML como base de nuestro almacenamiento, o a través de conectores específicos de bases de datos.

Por último, veremos cómo usar un ejemplo real de base de datos nativa como es BaseX, ejemplo que se puede usar tanto de forma local como en una arquitectura cliente/servidor.

INTRODUCCIÓN

El estándar XML puede que sea uno de los primeros mecanismos estandarizados que dentro de la informática moderna para el almacenamiento de datos de una forma estructurada.

A raíz de esa estandarización, surgió un sistema de gestión de esa información como son las bases de datos nativas XML, que proporcionan una gestión directa de la información a partir de documentos y colecciones de documentos del tipo XML.

Aunque hoy en día el uso de este tipo de bases de datos ha sido desplazado, tanto por las bases de datos relacionales como las bases de datos NoSQL, aún podemos ver ejemplos comerciales como eXist o BaseX que nos proporcionan mecanismos para gestionar y ofrecer información almacenada en XML.

CASO INTRODUCTORIO

En la empresa en la que trabajas como desarrollador, tienes un software un poco antiguo que se está migrando y desarrollando uno nuevo. Dicho software puede exportar todo su contenido almacenado en las bases de datos en formato XML.

Tu jefe le plantea al equipo la posibilidad de usar una Base de datos nativa XML para poder mantener esa información accesible durante la migración. Por ello, deberás decidir qué tipo de base de datos xml usarás, si existe alguna base de datos opensource que pudieras usar y si existe la posibilidad de usar esa base de datos dentro de un programa Java ya creado.

Al finalizar la unidad conocerás los diferentes tipos de bases de datos xml, las opciones de bases de datos xml nativas y utilizarás Java como lenguaje base para manejar bases de datos nativas xml.

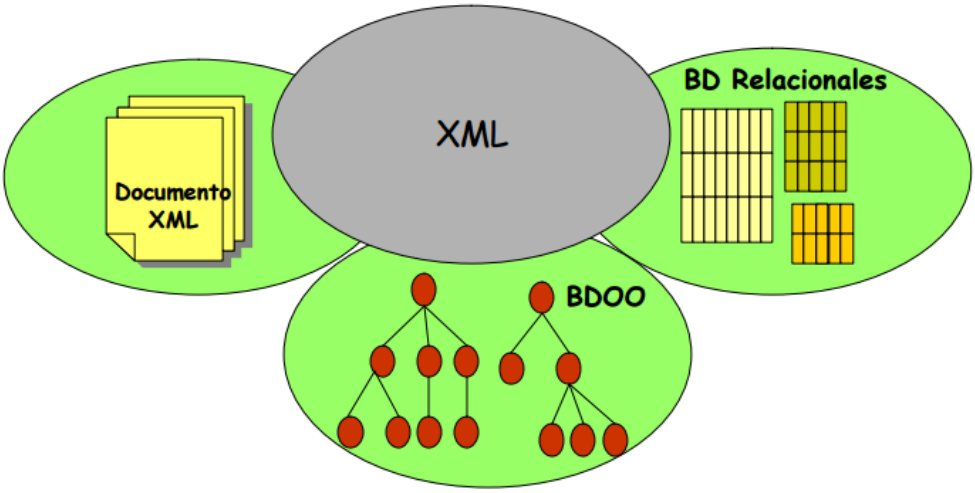
1. BASES DE DATOS NATIVAS XML

Tu jefe de departamento se plantea la necesidad de usar una serie de documentos XML exportados a partir de un viejo software de gestión empresarial que se está usando para mantener dicha información durante la migración.

Para llevar a cabo la migración de documentos, el primer paso es revisar las diferentes opciones del mercado que existen en el mercado sobre bases de datos XML, y se plantea que, en vez de almacenar la información en tablas, puede ser interesante desarrollarlo en una base de datos XML nativa opensource como eXist.

XML, eXtensible Markup Language ('lenguaje de marcas extensible'), es un lenguaje de marcas desarrollado por el World Wide Web Consortium (W3C), cuyo objetivo es almacenar datos en forma legible. Es un lenguaje derivado del lenguaje SGML, que permite definir la gramática de lenguajes específicos para estructurar documentos grandes.

La aplicación de XML no se limita exclusivamente a su uso en Internet, sino que se propone como un estándar para el intercambio de información estructurada entre diferentes plataformas. Como ejemplo se podría citar su uso en bases de datos, editores de texto, hojas de cálculo, etc.



Esquema base de datos XML
Fuente: Elaboración propia

XML es una tecnología bastante simple que tiene a su alrededor otras más completas que la complementan y la hacen mucho más grande y con unas posibilidades mucho mayores. Hoy en día es muy utilizada, ya que permite la compatibilidad entre sistemas para compartir la información de una manera segura, fiable y fácil.

Las **bases de datos XML nativas** son aquellas que tienen las siguientes características:

- Permiten definir el **modelo lógico de un documento XML** y almacenar y recuperar documentos según ese modelo.

- La **unidad mínima de almacenamiento** es el documento y estos se almacenan en colecciones.

- **Representan los elementos**, atributos y cualquier elemento de un documento XML como nodos.

- Soportan **lenguajes de consulta XML**.

- Permiten utilizar cualquier **estrategia de almacenamiento** de los documentos.

- Permiten la creación de **índices de los documentos XML**, objetos y colecciones que tienen almacenados para facilitar la realización de consultas de manera más rápida y eficiente.

- Asignan a cada documento XML un **identificador único** que permitirá reconocer al documento dentro del repositorio.

ENLACE DE INTERÉS

Accede para obtener más información sobre el lenguaje de marcado XML:

VÍDEO DE INTERÉS

Atiende a este vídeo para comprender mejor qué es el XML y su uso.

1.1 Comparativa con bases de datos relacionales

Las **bases de datos XML nativas** surgen por la imposibilidad de los SGBDR de manejar documentos XML sin sobrecargar de trabajo al gestor. Además, hasta su aparición era difícil reconstruir el documento original que había generado los datos. Con el surgimiento de las bases de datos nativas XML, es posible preservar el formato exacto del documento que genera los datos, manejar documentos sin estructurar y realizar consultas de manera más optimizada sobre estos.

Aunque el propósito de esta unidad es estudiar los SGBD XML nativos, se debe diferenciar entre **dos tipos de bases de datos XML**.

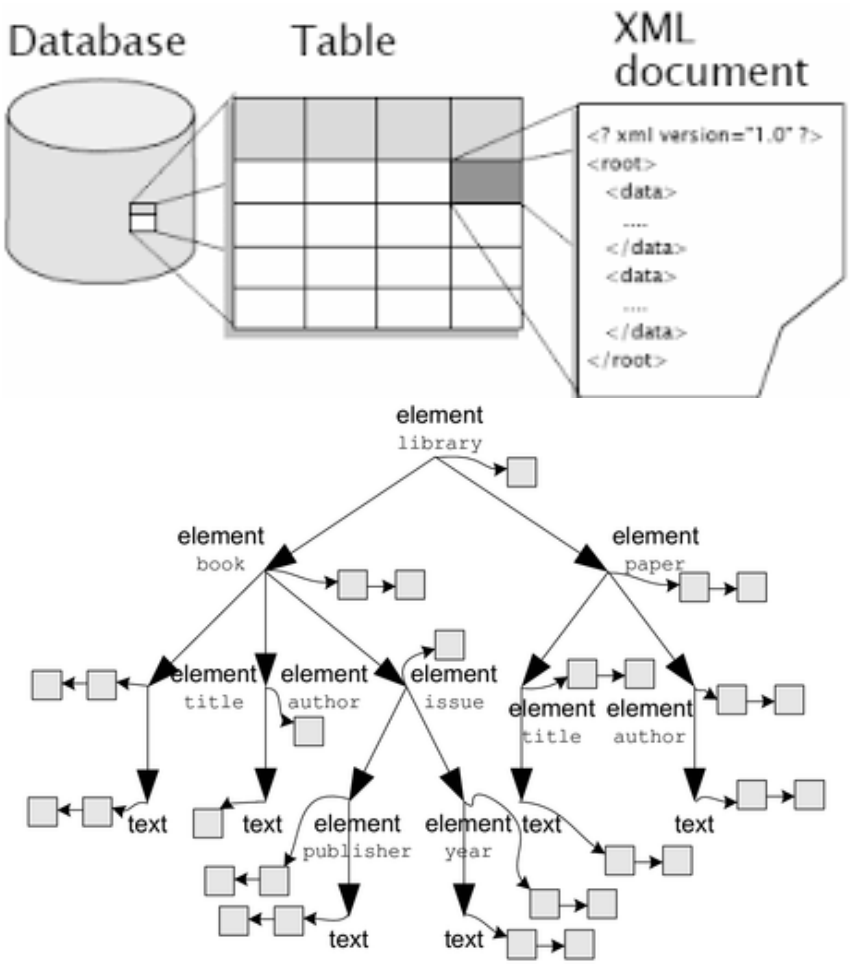
1. Las bases de datos **XML-enabled** son bases de datos que permiten XML, es decir, bases de datos que mapean el esquema del documento en un esquema de bases de datos y transfieren los datos según este mapeo.

2. Las bases de datos **XML nativas**, sin embargo, usan estructuras fijas que permiten almacenar cualquier documento XML de forma nativa.

La **principal diferencia** entre las bases de datos nativas XML y los sistemas XML-enabled es que las primeras tienen su propio modelo de datos y, sin embargo, las segundas tienen su propio modelo de datos e incorporan una capa software intermedia que permite almacenar documentos XML. Por tanto, necesitan un modelo previo sobre el que modelar la estructura existente en un tipo de documento XML en particular. Además, los SGBD XML-enabled solo pueden manejar y almacenar los documentos que encajan dentro de su modelo, mientras que los SGBD XML nativo pueden manejar todos los documentos posibles.

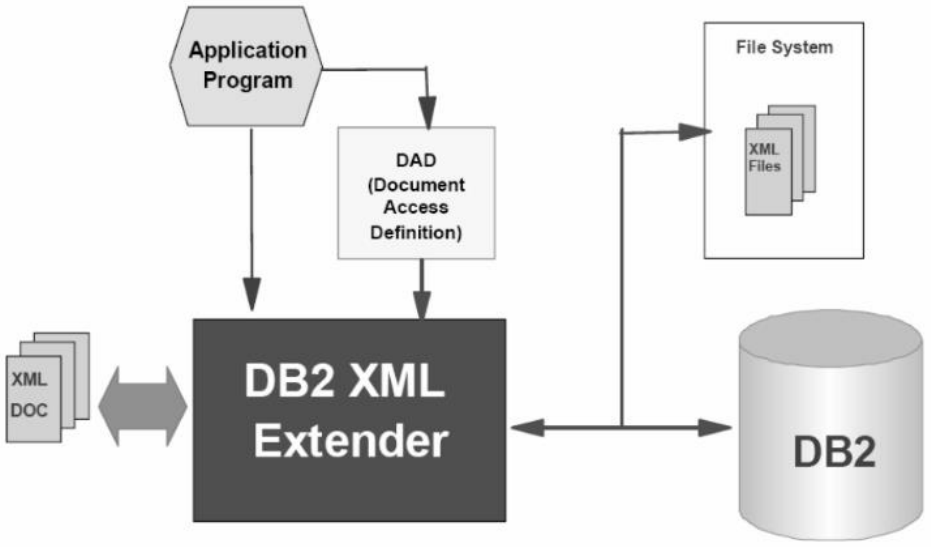
Se debe utilizar **una base de datos XML-enabled** cuando se quieran publicar datos existentes como XML o importarlos desde un documento XML en una base de datos existente, no para almacenar documentos XML completos.

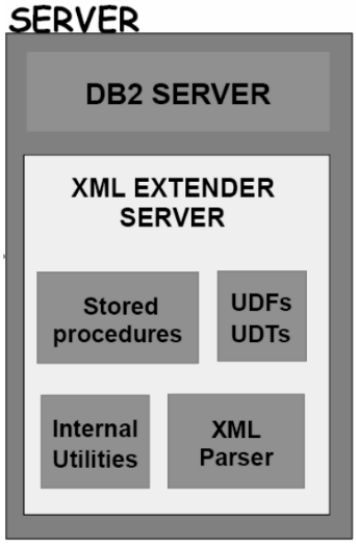
De **manera gráfica** la diferencia entre ambos tipos de bases de datos sería la siguiente:



Diferencias entre base de datos XML y nativa XML
Fuente: https://bbddxml.wordpress.com/tag/sedna-xml-database/

Como ejemplo de SGBD XML-enabled se podrían citar Oracle y DB2, entre otros. A continuación, se muestra la **arquitectura del SGBD DB2**:





Arquitectura SGBD DB2

Fuente: Elaboración propia

PARA SABER MÁS

Si quieres saber más sobre estas bases de datos, accede a este documento:

1.2 Principales gestores comerciales y libres

Son muchos los SGBD XML existentes. En este cuadro comparativo se pueden observar divididos en función de su tipo de almacenamiento:

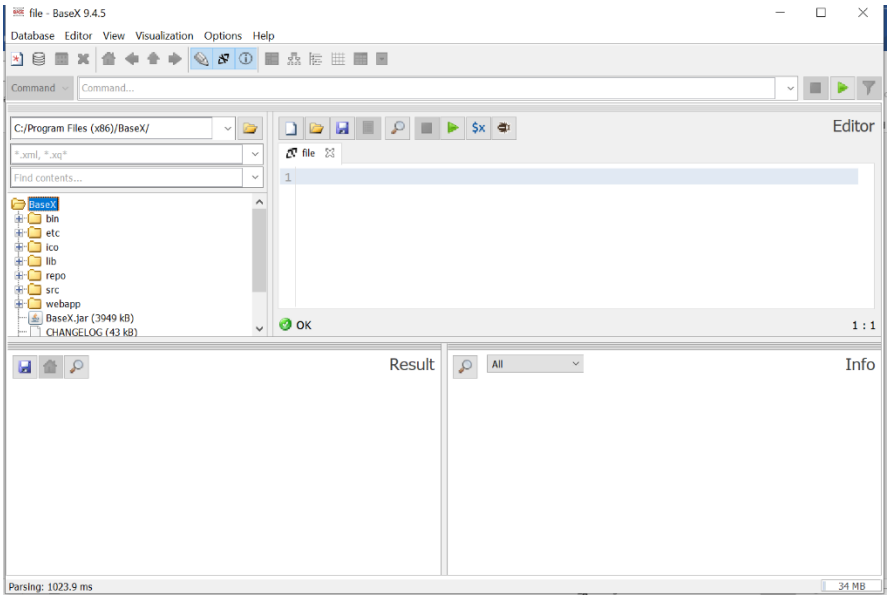
	BD XML nativas	EXTENSIONES DE DB		
		Almacenamiento no estructurado	Almacenamiento estructurado	Mapeo
Comercial	eXcelon XIS GoXML DB Infonyte-DB Tamino TEXTML X-Hide/DB	IBM DV2 SML Extender Microsoft SQLXML Oracle XML DB		Oracle XML DB/Mapeo estructurado
Código abierto	dbXML eXist Xindice		Ozone/XML (OO)	
Investigación	Lore Natix PDOM TIMBER		Monet XML Shimura et al. XML Cartidge	

Tabla comparativa Bases de datos XML
Fuente: Elaboración propia

Esta unidad se centrará en los dos SGBD XML nativos más utilizados, uno libre y otro comercial.

- **BaseX**. Es un framework ligero y escalable, tal y como la documentación y el proveedor nos indica en su documentación de referencia.

  Nos permite crear, configurar y procesar información a través del estándar XQuery 3.1., además de proporcionarnos un interfaz de desarrollo y administración. Para comenzar a trabajar con Base X, simplemente descargaremos la última versión, comprobaremos los requisitos de Java instalados y ejecutaremos el servidor y administrador.



Interfaz de BaseX

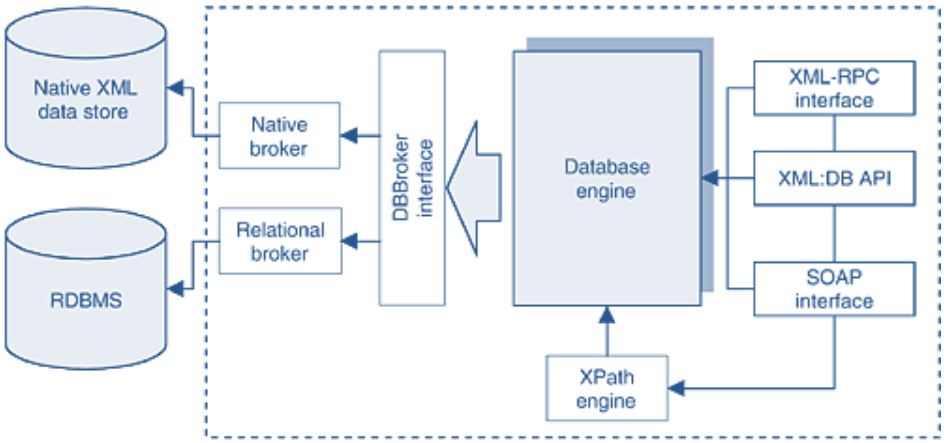
Fuente: Elaboración propia

- **eXist.** SGBD XML nativo **libre y de código abierto** que cuenta con las funciones básicas de un SGBD de su categoría, además de algunas características más avanzadas como búsqueda de términos, etc.

Para acceder a los datos que almacena se suele utilizar la API **XML:DB** para el lenguaje de programación Java. Entre sus principales características cabe destacar:

- Soporte para distintos lenguajes de consulta XML: **XQuery, XPath y XSLT**.
- Indexación de documentos.
- Soporte para la actualización de los datos.
- Soporta multitud de protocolos de acceso: SOAP, XML-RPC, WebDav, etc. En el siguiente punto se profundizará en cada uno de ellos.

Los componentes principales de este SGBD son:



Componentes principales eXist.db

Fuente: Elaboración propia

ENLACE DE INTERÉS

Visita la web oficial de este SGBD, ya que ahí se puede encontrar multitud de documentación, ejemplos y, lo más importante, una demo online para probar sus funciones:

PARA SABER MÁS

Para ampliar tus conocimientos en BaseX, accede a esta web:

1.3 Estrategias de almacenamiento

Antes de abordar las distintas estrategias de almacenamiento con las que se puede contar, es necesario tener clara la diferencia entre los tipos de documentos existentes:

DOCUMENTOS CENTRADOS EN DATOS

- Son documentos para el intercambio de datos, generalmente entre maquinas.
- Suelen ser documentos con estructuras regulares y bien definidas.
- Los distintos datos que transmiten son partículas atómicas bien definidas.
- Tendrán como origen o destino una base de datos.

DOCUMENTOS CENTRADOS EN CONTENIDO

- Documentos con una estructura irregular (aunque posean formato no es tan estricto y definido, decimos en este caso que son semiestructurados).
- El origen y destino de este tipo de documentos suelen ser personas, y suelen ser creados a mano.

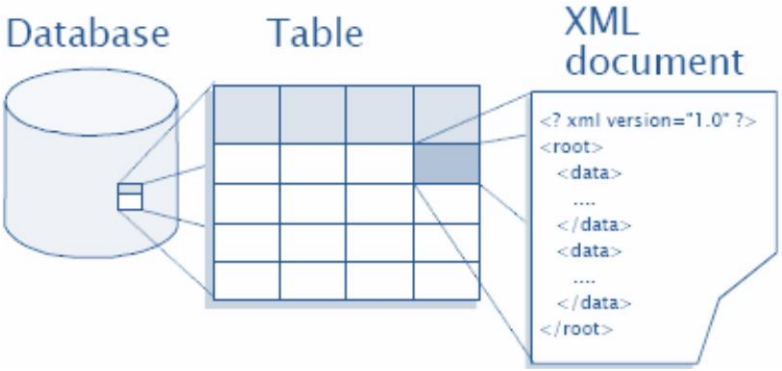
Ahora sí podemos pasar a analizar las distintas **estrategias de almacenamiento** disponibles, que son:

- En fichero.
- En una base de datos relacional.
- En una base de datos XML.

En cuanto a las diferencias, serían las siguientes:

- **Almacenamiento en un fichero.** Esta opción limita a la hora de realizar operaciones sobre el contenido, por lo que es la menos recomendada.

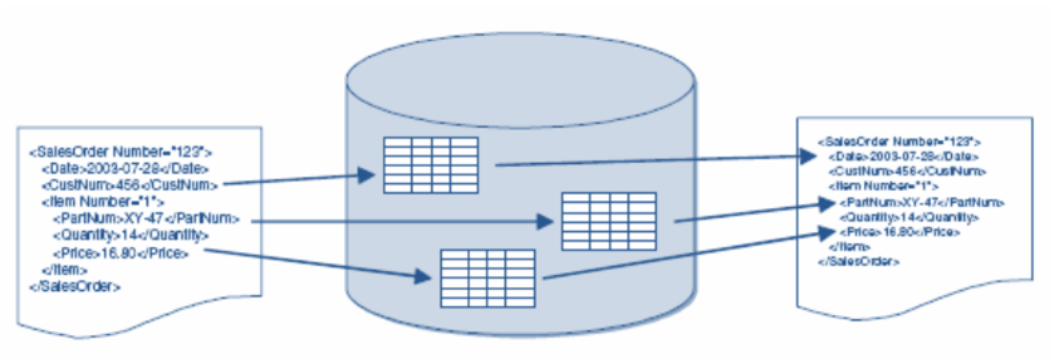
- **Almacenamiento en una base de datos relacional.** Permite almacenar únicamente los datos que interesa conservar, pero se pierden determinadas partes de un documento que pueden ser necesarias a la hora de reconstruir el documento a partir de los datos almacenados. En este caso se pueden almacenar los documentos directamente en un campo de la tabla, tal cual muestra la figura. Con la ventaja de que mantiene el documento intacto, pero no se pueden realizar operaciones de consulta sobre él.



Almacenamiento en base de datos relacional

Fuente: Elaboración propia

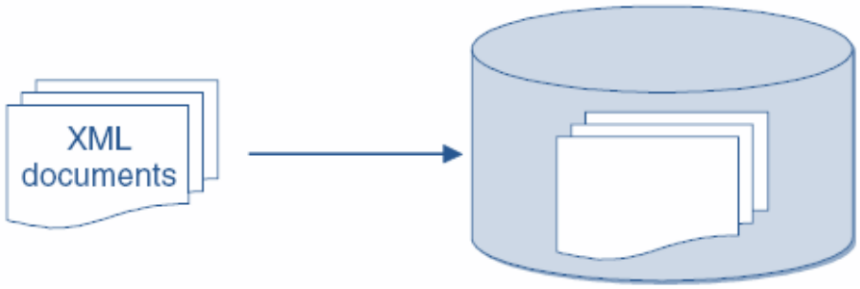
- Otra opción es **almacenar la información del documento en tablas** donde cada fila del documento será almacenada como una tupla de la tabla y cada dato de la fila se corresponderá con un campo de la tabla. Una última opción dentro del almacenamiento en bases de datos sería almacenar los datos del documento como objetos serializados, de manera que cada objeto se almacena en una tabla y las propiedades de este son las columnas de la tabla.



Almacenamiento en base de datos relacional

Fuente: Elaboración propia

- **Almacenamiento sobre una base de datos XML.** Esta es la opción más recomendable y la que se va a estudiar en esta unidad.



Almacenamiento en base de datos XML

Fuente: Elaboración propia

1.4 Colecciones y documentos

Como hemos estudiado en anteriores apartados, las **bases de datos XML** almacenan la información en documentos XML que tendrán una estructura y por lo tanto una información.

Además de este almacenamiento básico y esencial de la información, muchas bases de datos XML permiten la agrupación de estos documentos en lo que se denomina "**colecciones**". Es de alguna forma una categorización de los documentos que permite a las bases de datos gestionar un conjunto de documentos simultáneamente.

Las colecciones pueden presentar una jerarquía de colecciones, de igual forma a como tendríamos ordenado un sistema de ficheros y directorios dentro de nuestro sistema operativo.

Un ejemplo lo tenemos con la base de datos de estudio **eXist.db**. Esta base de datos soporta el concepto de colecciones siendo agrupamientos de tanto documentos XML, documentos binarios u otro tipo de colecciones de acuerdo con la documentación oficial.

En la base de datos eXist **organiza los documentos en colecciones** como si fueran directorios teniendo el root definido en /db, y por debajo las colecciones. Estas colecciones son de dos tipos:

- Ficheros que son denominados recursos.
- Sub-colecciones, que son agrupaciones de ficheros o documentos para una organización de la información.

1.5 Ventajas y Desventajas de las Bases de Datos Nativas XML

Las bases de datos nativas XML presentan una serie de **ventajas** significativas frente a otros sistemas de gestión de bases de datos:

- **Almacenamiento eficiente de documentos XML:** al estar diseñadas específicamente para manejar documentos XML, permiten almacenar y recuperar datos sin necesidad de transformaciones o mapeos intermedios.

- **Preservación de la estructura y orden del documento:** mantienen intacta la estructura jerárquica y el orden de los elementos tal como aparecen en el documento original.

- **Soporte nativo para lenguajes de consulta XML**: soportan de forma nativa lenguajes de consulta como XQuery y XPath.

- **Flexibilidad en el esquema de datos:** no requieren un esquema fijo para almacenar los datos, lo que las hace ideales para manejar información semi-estructurada o con estructuras variables.

A pesar de las ventajas mencionadas, las bases de datos nativas XML también presentan algunas **desventajas**:

- **Rendimiento limitado en grandes volúmenes de datos**: aunque son excelentes para manejar documentos XML, pueden no ser tan eficientes como las bases de datos relacionales o NoSQL para manejar grandes volúmenes de datos.

- **Complejidad de uso:** el manejo de consultas resulta más complejo con lenguajes como XQuery o XPath.

- **Compatibilidad limitada**: las bases de datos nativas XML no están tan adoptadas como otros sistemas de gestión de bases de datos (SGBD).

ENLACE DE INTERÉS

La documentación oficial a la que hacemos referencia y que estamos usando y traduciendo al castellano la podemos encontrar en:

EJEMPLO PRÁCTICO

En el departamento donde trabajas, estás evaluando diferentes opciones de bases de datos nativas XML para realizar ese servicio de información a partir de los datos antiguos exportados.

Algunas de las características que necesitas son:

Opensource.
Multiplataforma.
Integración con desarrollo Java.
Arquitectura cliente/servidor.

¿Qué base de datos debes elegir?

Reduciendo nuestra búsqueda y estado del arte de las posibles opciones de bases de datos nativas XML, te quedas con eXistDb y con BaseX. Respecto a ambas soluciones:

- Ambas soluciones son opensource.
- Ambas soluciones son multiplataforma.
- Ambas soluciones permiten la integración y desarrollo con Java.

A partir de esas características comunes, y si visitas las características de ambas, ves que BaseX sí que ofrece la opción de funcionar mediante una arquitectura cliente/servidor.

Por lo tanto, la opción elegida sería BaseX.

2. API JAVA ESTÁNDARES PARA BD-XML NATIVAS

Una vez elegida la herramienta de gestión de base de datos, y antes de comenzar a trabajar con ella, tu jefe se plantea cómo se podría modificar el software actual basado en tecnología Java para el uso de bases de datos XML. Sabes que, para desarrollarlo XML en Java, puedes usar la API de XQJ que garantiza un uso más flexible de la información almacenada en formato XML.

Los **datos almacenados en las BD** tienen que poder ser accesibles desde aplicaciones desarrolladas en diferentes lenguajes y, por este motivo, los SGBD se ven obligados a facilitar interfaces de programación para los lenguajes de programación más empleados. Los fabricantes de SGBD, conocedores de la tecnología empleada en su producto, facilitan una API para permitir el acceso, desarrollada de la manera más eficiente posible para su producto, hecho que comporta la aparición de un problema: la API proporcionada para cada SGBD es propia y diferente de las API de los otros SGBD y, en consecuencia, las aplicaciones desarrolladas quedan ligadas al SGBD y los programadores tienen que conocer un montón de API diferentes, tantas como número de SGBD diferentes tengan que enlazar.

Ante la anarquía de APIO existente para atacar los SGBD de un determinado tipo, acostumbran a aparecer intentos para estandarizar el mecanismo y proporcionar una API estándar. En el caso de las BD-XML nativas ha habido dos procesos de estandarización para el lenguaje Java, que han dado lugar a dos API estándares:

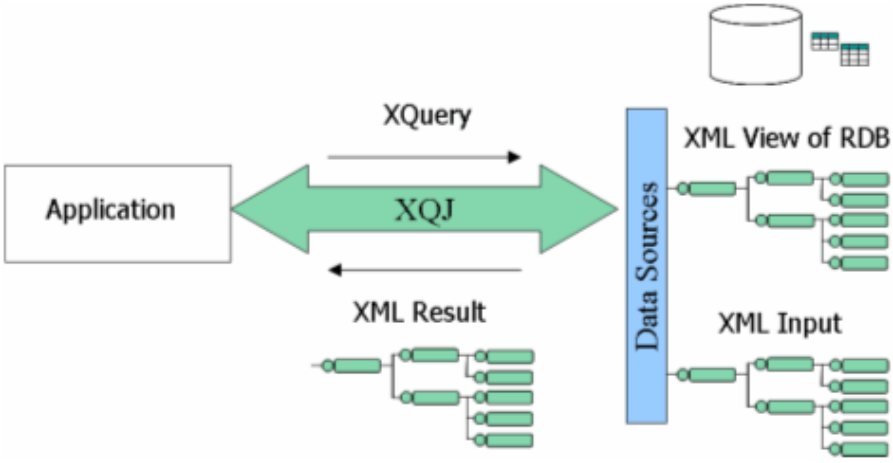
- **XML:DB** (también llamada CHAPE).
- **XQueryAPI** for Java (XQJ).

Actualmente hay muchos SGBD-XML nativas que facilitan la implementación de las dos API. Pero la API XQJ es la que está destinada a evolucionar, dado que está promovida por el W3C, mientras que la API XML:DB, anterior en el tiempo, está parada desde el 2003. Por lo tanto, se puede considerar ya obsoleta.

2.1 API XQJ

XQuery API for Java (XQJ) es una interfaz de programación de aplicaciones Java pensada para utilizar el lenguaje XQuery para obtener información de BD-XML nativas, de manera parecida a cómo JDBC es una API pensada para utilizar el lenguaje SQL para acceder a BDR.

Numerosos SGBD-XML nativas facilitan la conectividad desde Java intermediando esta XQJ, de forma que podemos desarrollar aplicaciones que accedan a SGBD-XML nativas vía XQJ con la única particularidad de tener que utilizar la implementación de la API que facilita cada SGBD.



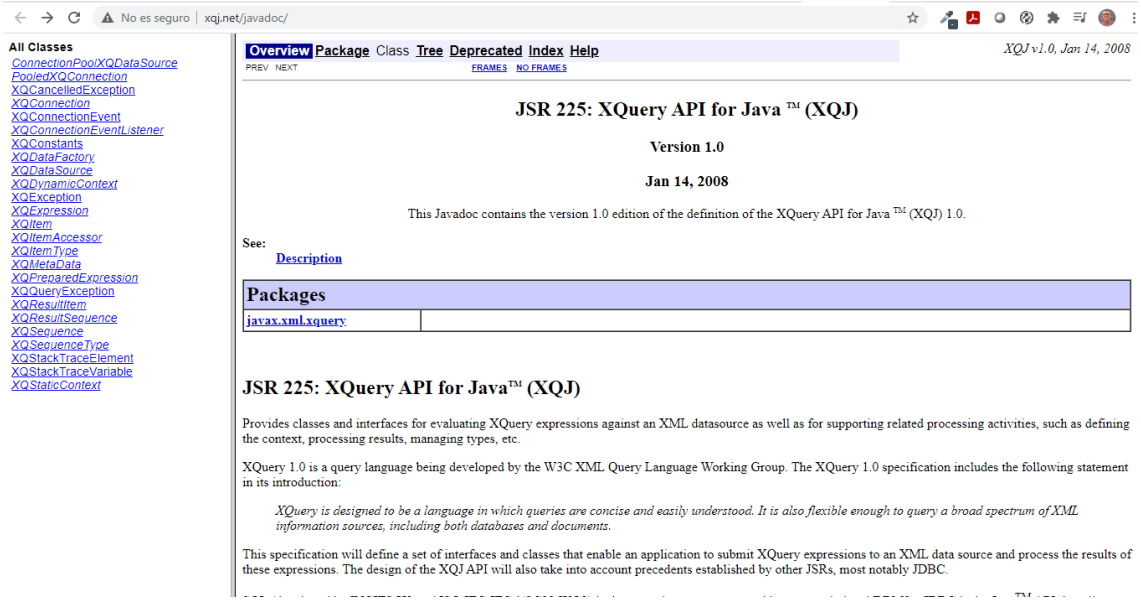
Esquema funcionamiento XQJ
Fuente: Elaboración propia

Los principales **objetivos** que se persiguen son:

- Definir un estándar que sea independiente del fabricante.
- Dar soporte al estándar XQuery.
- Facilidad de uso.
- Fomentar su uso desde el lenguaje Java.

Si damos un vistazo a la **documentación de la API XQJ**, veremos que está constituida por un gran número de interfaces y unas pocas clases. Esto es así porque cada SGBD tiene que proveer las clases que implementan las interfaces que dictamina el API.

En el desarrollo de aplicaciones con esta API, utilizaremos siempre **referencias** a estas **interfaces** y nunca utilizaremos directamente referencias en las clases suministradas por el SGBD. De este modo conseguiremos diseñar aplicaciones que se puedan utilizar para gestionar BD en varios SGBD que implementan el APIO XQJ.



API de XQuery

Fuente: http://xqj.net/javadoc/

2.2 Establecimiento y cierre de conexiones

Las aplicaciones que acceden a BD necesitan, como primer paso para poder gestionar los datos de la BD, establecer la conexión con la BD a gestionar. La API XQJ facilita dos interfaces apropiadas para conseguir este objetivo:

- **XQDataSource**, fábrica para obtener objetos **XQConnection**.
- **XQConnection**, para referenciar conexiones (sesiones) con un SGBD específico. Toda conexión se logra, forzosamente, a través de un objeto **XQDataSource**.

Así, para obtener una conexión, hay que seguir el **esquema** siguiente:

```
XQDataSource xqs = new ClasseEspecíficaQueGeneraXQDataSource();
XQConnection con = xqs.getConnection(listaParámetros);
```

La primera línea del código anterior muestra cómo crear un objeto *XQDataSource* específico del SGBD al cual nos queremos conectar. La segunda línea muestra como establecer la conexión y, como lista de parámetros, tendremos que incluir los adecuados en el SGBD.

Por cada SGBD-XML nativas con que queramos conectar vía XQJ, hay que saber la clase específica facilitada por el SGBD que implementa la interfaz *XQDataSource*. Por ejemplo, para **conectarnos a eXist.db**:

```
XQDataSource xqs = new net.xqj.exist.ExistXQDataSource();
XQConnection con = xqs.getConnection(listaParámetros);
```

El código anterior es código específico para el SGBD eXist.db, y nos interesa, si es posible, escribir código que sea independiente, en el mayor grado posible, del SGBD.

Por lo tanto, nos interesa obtener el **objeto XQDataSource de forma genérica**, sin llamar a ningún método específico de la API suministrada por el SGBD. Para conseguirlo escribiremos el siguiente:

```
String driver = "nombreClaseEspecíficaQueGeneraXQDataSource";
XQDataSource xqs = (XQDataSource)Class.forName(driver).newInstance();
XQConnection con = xqs.getConnection(listaParámetros);
```

Una vez se dispone del objeto *XQDataSource* ya estamos en condiciones de crear un **objeto XQConnection** para establecer la sesión de trabajo con el SGBD y esto lo conseguimos con el método getConnection() de la interfaz *XQDataSource*. La API XQJ facilita tres sobrecargas del método *getConnection()*:

```
XQConnection getConnection(java.sql.Connection con)
throws XQException;
XQConnection getConnection() throws XQException;
XQConnection getConnection(java.lang.String username,
java.lang.String passwd)
throws XQException;
```

Las tres **modalidades** intentan establecer una conexión contra el SGBD de diferentes maneras:

A. Utilizando una conexión JDBC ya existente.
B. Sin aportar ninguna información (parámetro).
C. Indicando el usuario y la contraseña (parámetros).

Si hemos trabajado con SGBD relacionales estamos acostumbrados a indicar, cuando queremos establecer la conexión, entre otros parámetros: la máquina (IP o nombre), el puerto, la base de datos, el usuario y la contraseña. Observando las modalidades anteriores, estaremos de acuerdo que las opciones segunda y tercera parecen incompletas, puesto que no indicamos algunos de los parámetros claves: máquina y puerto donde reside el SGBD y nombre de la base de datos. A la segunda opción, incluso, no se indican usuario y contraseña.

La solución pasa para saber que la interfaz *XQDataSource* facilita el método *setProperty()*, que nos permite definir una serie de propiedades antes de intentar establecer la conexión con el **método *getConnection()***:

```
void setProperty(java.lang.String name,
java.lang.String value)
throws XQException;
```

Entre las propiedades que permite establecer el método *setProperty()*, encontramos el nombre de la máquina, el puerto, el nombre de la base de datos, el usuario y la contraseña. No todos los SGBD-XML tienen las mismas propiedades o utilizan idénticos nombres para una misma propiedad. Hay que saber, pues, las propiedades que cada SGBD admite para poderlas asignar con el método *setProperty()* antes de establecer la conexión con el método *getConnection()* y para conseguirlo, la **interfaz *XQDataSource*** facilita el método siguiente:

```
java.lang.String[] getSupportedPropertyNames();
```

Por último, para el cierre de una conexión tendremos el **método *close()***, aunque debemos tener en cuenta los recursos abiertos durante las transacciones, que deberán ser cerrados, tales como resultados, ficheros u otros.

```
result.close();
expr.close();
conn.close();  // cierra la conexión y libera todos los recursos
```

2.3 Realización de consultas. Clases y métodos

Todos los SGBD acostumbran a facilitar dos mecanismos de ejecución de sentencias contra la BD, utilizando el lenguaje que corresponda (SQL en SGBDR y SGBDOR, OQL/OML en SGBDOO y XQuery/Update en SGBD-XML):

1. **Ejecución inmediata de una sentencia**, utilizada cuando la sentencia se tiene que ejecutar una única vez.

2. **Ejecución de sentencias paramétricas o preparadas**, utilizada cuando la misma sentencia (o sentencia similar con algunos valores diferentes) se tiene que ejecutar repetidamente y consiste a escribir la sentencia como una plantilla que contiene algunas variables que se irán sustituyendo a cada ejecución.

La API XQJ facilita los dos mecanismos a través de las interfaces:

- **XQExpression**, para la ejecución inmediata de sentencias.
- **XQPreparedExpression**, para la ejecución de sentencias paramétricas.

Centrémonos, en este momento, en la ejecución inmediata de sentencias proporcionada por la interfaz *XQExpression*. Para ejecutar una sentencia de manera inmediata, crearemos un objeto *XQExpression* a partir del **método *XQConnection.createExpression().*** Disponemos de dos sobrecargas de este método:

```
XQExpression createExpression() throws XQException;
XQExpression createExpression(XQStaticContext properties)
throws XQException;
```

En ambas sobrecargas se obtiene un objeto *XQExpression* que podremos utilizar para ejecutar **sentencias XQuery/Update** de manera inmediata. La segunda sobrecarga permite indicar las propiedades del contexto estático que se tendrán en cuenta al evaluar la expresión, mientras que la primera sobrecarga toma como contexto estático el asociado a la conexión.

Una vez tengamos el objeto *XQExpression*, podremos a través de él ejecutar consultas y otras órdenes de manera inmediata. Recordamos que a XQuery hay que distinguir, como SQL, las sentencias "consulta" que pueden devolver un conjunto de resultados que habrá que procesar, de las sentencias "no consulta" que permiten ejecutar una orden (inserción, eliminación o actualización e incluso órdenes específicas del SGBD), de la cual se nos informa, como mucho, del éxito o fracaso de su ejecución. Por este motivo la interfaz ***XQExpression distingue los métodos executeQuery*** para las "consultas" de los métodos *executeCommand* para las "no consultas":

```
void executeCommand(java.lang.String cmd)throws XQException;
void executeCommand(java.io.Reader cmd) throws XQException;
XQResultSequence executeQuery(java.lang.String query)
throws XQException;
XQResultSequence executeQuery(java.io.Reader query)
throws XQException;
XQResultSequence executeQuery(java.io.InputStream query)
throws XQException;
```

Además de los métodos anteriores, disponemos del **método *close()*** para cerrar la expresión, liberando todos los recursos asociados cuando ya no sea necesaria. La ejecución del método *close()* sobre la conexión cierra todas las expresiones definidas sobre ella. Fijémonos que el método *executeQuery()* devuelve, si todo va bien, un objeto *XQResultSequence*, interfaz que deriva de la interfaz *XQSequence*, la cual nos facilita un conjunto de métodos para evaluar la secuencia de resultados obtenidos con el método *executeQuery().*

La interfaz XQResultSequence representa una secuencia de elementos siguiendo la **definición de XDM** (XQuery 1.0 and XPath 2.0 Fecha Modelo, de W3C). Un vistazo a la documentación de esta interfaz nos muestra que disponemos de métodos por:

- **Procesar sus elementos:** next(), previous(), getItem(), getPosition(), count(), first(), last(), isFirst(), isLast(), isBeforeFirst(), isAfeterLast() y un conjunto de métodos get para obtener el contenido de un elemento en el formato adecuado (getBoolean(), getByte(), getDouble()…).

- Obtener una versión seriada de la secuencia como un objeto *String (método getSequenceAsString()).*

EJEMPLO PRÁCTICO

En el departamento donde trabajamos queremos comprobar si se puede usar la API XQJ independientemente de las librerías proporcionadas por BaseX para el uso de sentencias XQuery. ¿Es posible?

Para ello necesitaremos el driver adecuado de conexión contra Basex y seguir los pasos de conexión y uso de Xquery:

1. **El primer paso sería el uso del driver y la conexión contra la base de datos:**

```
String DRIVER = "net.xqj.basex.BaseXXQDataSource";

XQDataSource source = (XQDataSource) Class.forName(DRIVER).
        getDeclaredConstructor().newInstance();
```

2. **Una vez que ya tenemos realizada la conexión y usando los privilegios de administrados, abrimos alguna base de datos que tengamos:**

```
XQConnection conn = source.getConnection("admin", "admin");

String input = System.getProperty("user.dir") +
"/src/main/resources/xml/input.xml";
```

3. **A partir de aquí realizaríamos las consultas que deseásemos:**

```
XQPreparedExpression expr = conn.prepareExpression("doc('" + input +
"')//li");
QResultSequence result = expr.executeQuery();

    while(result.next()) {
      // Imprimimos los resultados
      System.out.println(result.getItemAsString(null));
    }
```

4. **Por último, cerraríamos los recursos:**

```
expr.close();
```

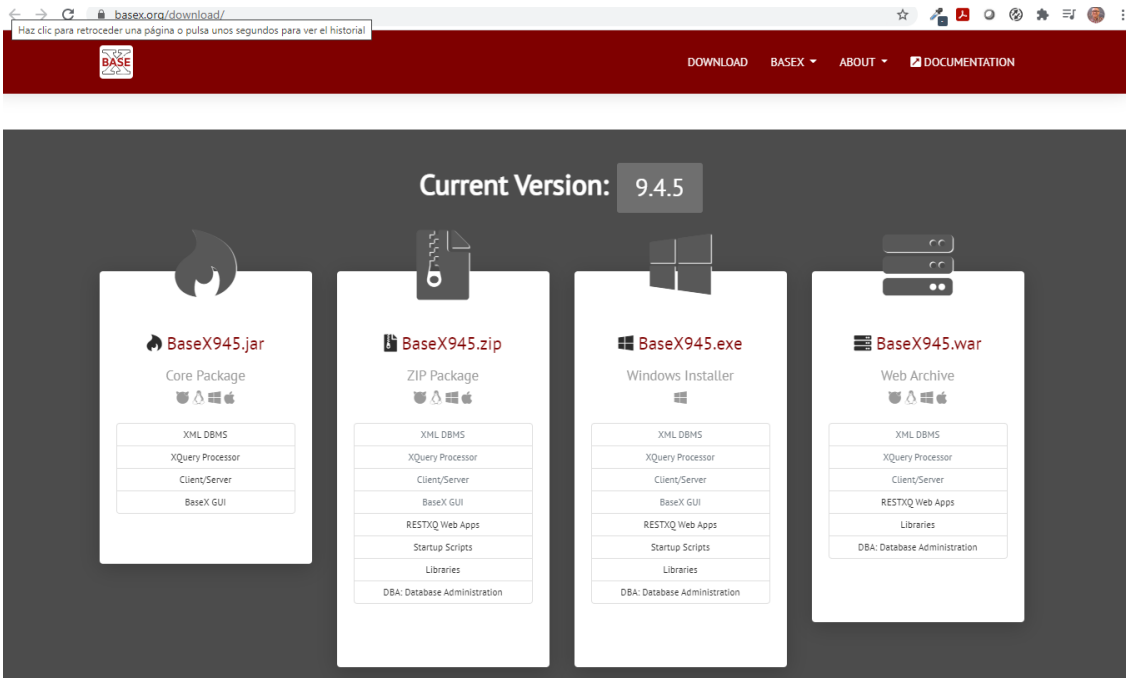
3. USO Y CREACIÓN DE BASES DE DATOS XML CON BASEX

Finalmente has elegido BaseX como herramienta de desarrollo dentro del departamento, ya que no sólo permite realizar las lecturas contra los recursos de la información exportada, sino que permite trabajar con varias arquitecturas como la de cliente/servidor.

A partir de esta decisión, usarás su interfaz para operar sobre los documentos/colecciones y luego crearás un proyecto en Java que sirva para gestionar la información almacenada en los XML.

En la **comparación y presentación** de ejemplos sobre bases de datos nativas XML, vimos como BaseX es un ejemplo fantástico y ligero para introducirnos con las bases de datos nativas sobre XML, además de proporcionarnos herramientas de desarrollador que nos ayuda en el proceso de administración de la base de datos.

Aunque lo introdujimos en el punto 1, la instalación de BaseX es muy sencilla, y dependiendo del sistema operativo en el que trabajemos, elegiremos uno u otro paquete tal y como vemos en la imagen del fabricante.

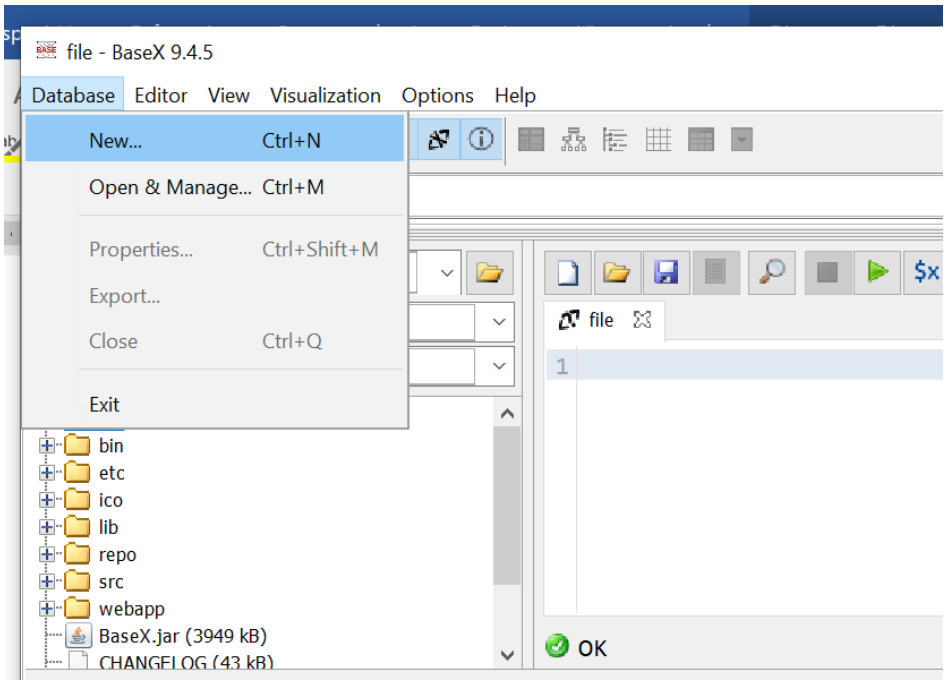


Instalación BaseX

Fuente: https://basex.org/download/

3.1 Interfaz y creación de una base de datos

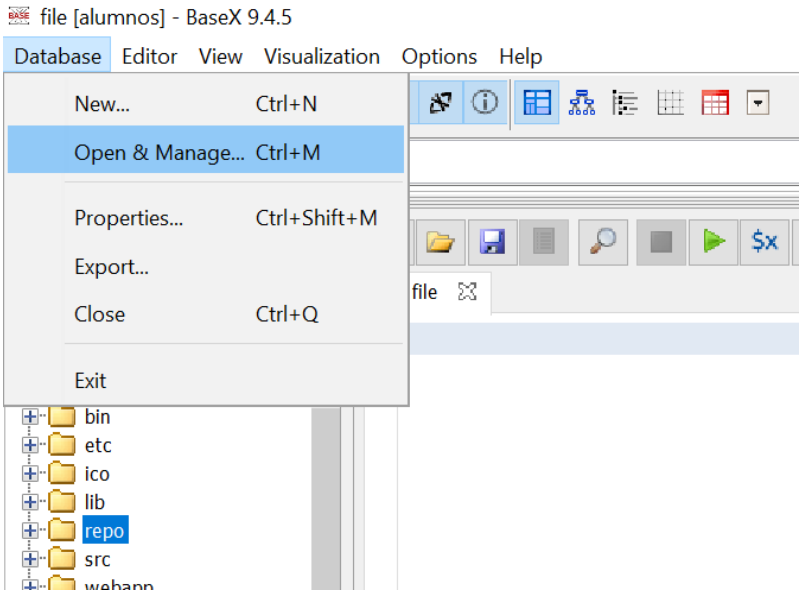
Para la creación de una nueva base de datos usaremos el interfaz, para ello a través del menú iremos al apartado *Database->New:*



Nueva base de datos

Fuente: Elaboración propia

Una vez creada, podemos abrir y administrarla a través del menú Open&Manage:



Administrar base de datos

Fuente: Elaboración propia

PARA SABER MÁS

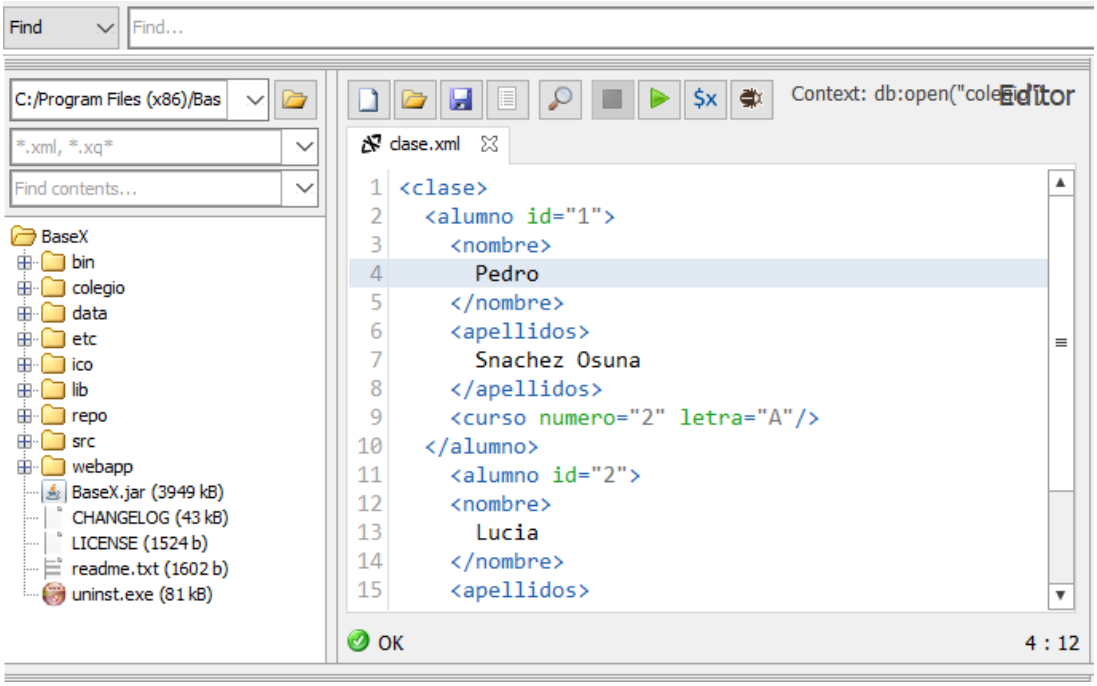
Profundiza un poco más sobre BaseX:

VÍDEO DE INTERÉS

Mediante este vídeo comprobarás cómo comenzar a usar BaseX:

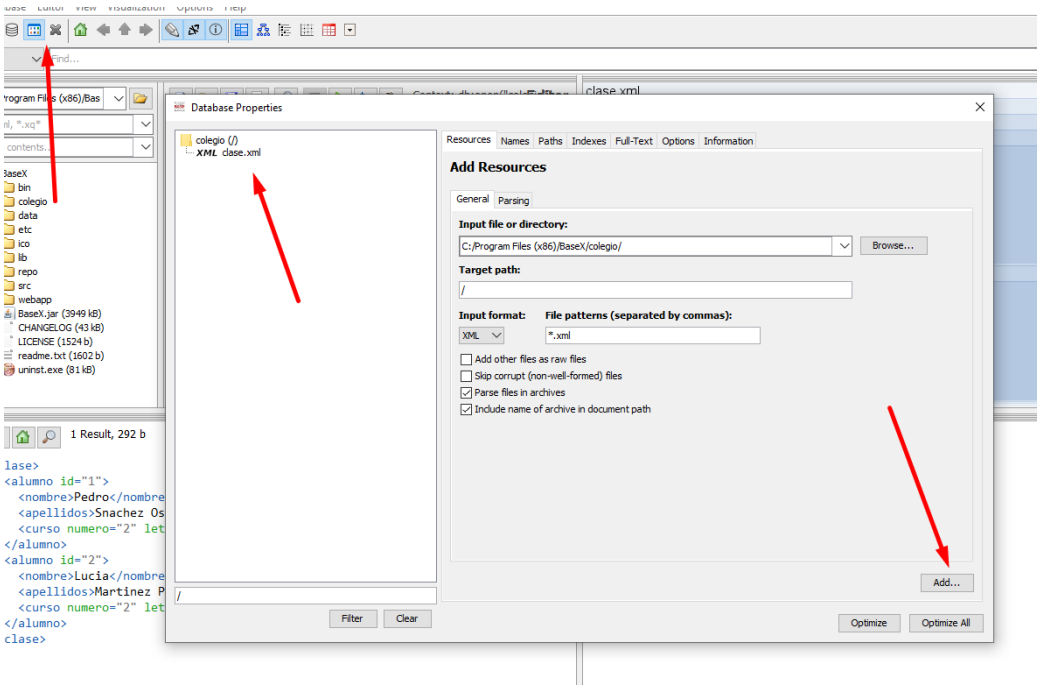
3.2 Añadir, modificar y eliminar documentos

Para añadir nuevos documentos a nuestra base de datos, podemos usar el **interfaz de edición** dentro de nuestro árbol de proyecto tal y como observamos en la imagen:



Nuevo documento
Fuente: Elaboración propia

En dicho interfaz, comenzaremos a escribir nuestro documento XML y que una vez finalizado guardaremos, a poder ser, dentro del mismo directorio de creación de la base de datos. Para que dicho documento se añada a nuestra base de datos, podemos a través de las propiedades de la base de datos añadirlo tal y como observamos en la imagen:



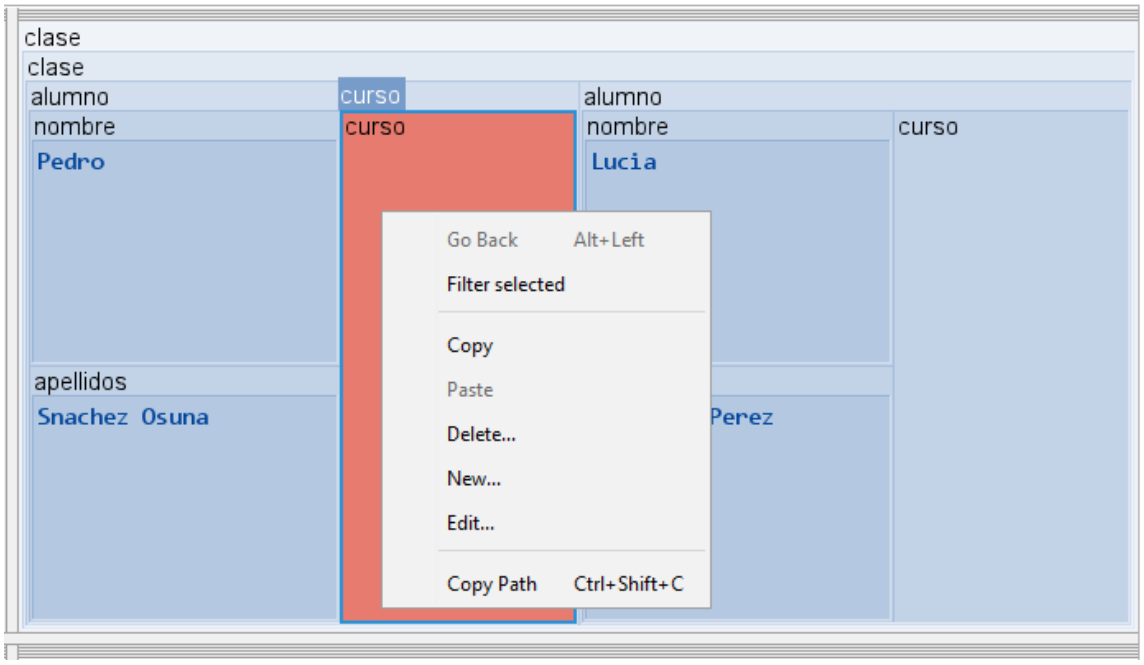
Interfaz de propiedades
Fuente: Elaboración propia

Las acciones una vez añadido que podemos realizar son:

- Para eliminar este documento, simplemente haciendo click derecho sobre el recurso podremos eliminarlo.

- Renombrar el fichero, también a través del menú de propiedades, haciendo click derecho podremos cambiar el nombre.

- Modificar el contenido a través del editor o del interfaz de mapeado.

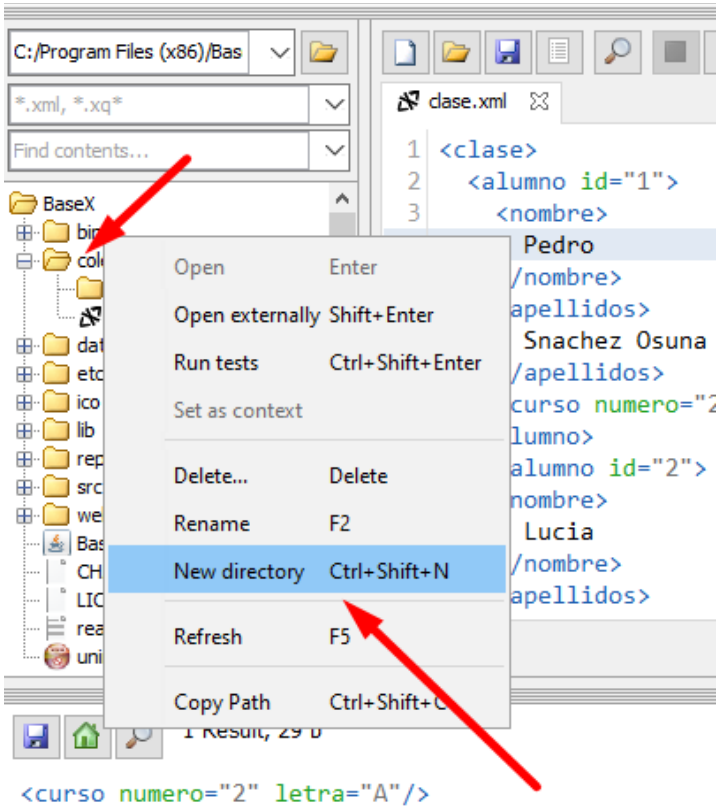


Interfaz de mapeado
Fuente: Elaboración propia

3.3 Creación y borrado de colecciones

No existe una capa específica para la creación y mantenimiento de las colecciones dentro de BaseX, ya que estas colecciones se mantienen dentro de directorios dentro de la misma base de datos creada. Así pues, cuando queramos crear una colección de documentos o ficheros XML realizaremos los siguientes pasos:

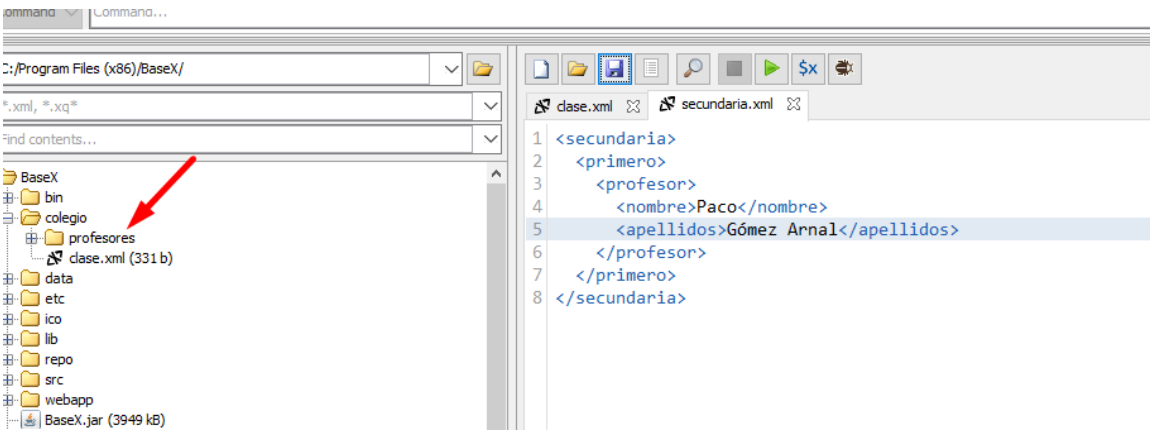
1. Desde el explorador de BaseX, generaremos un nuevo directorio dentro del directorio raíz de nuestra base de datos.



Nueva colección

Fuente: Elaboración propia

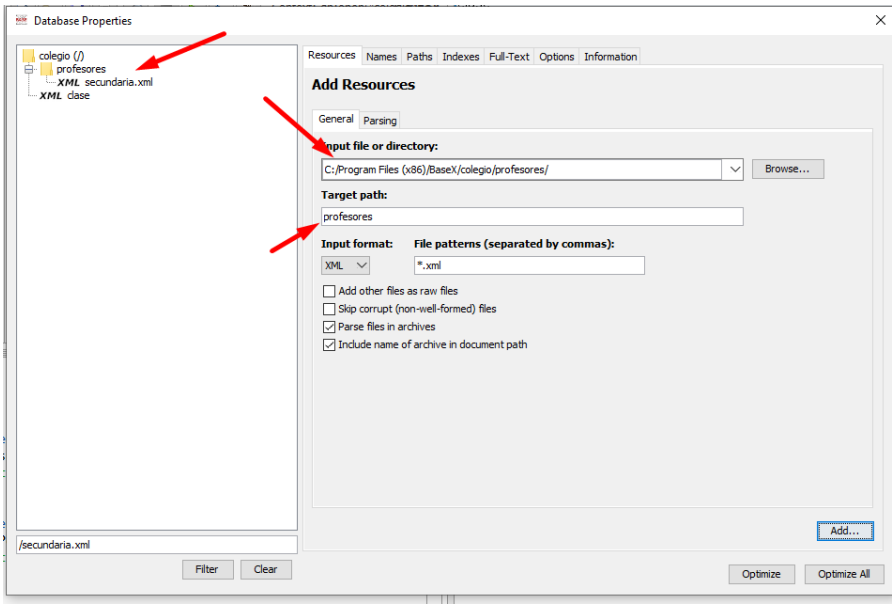
2. Crearemos los documentos o ficheros que queramos añadir a esta colección.



Nuevos documentos dentro de la colección

Fuente: Elaboración propia

3. Por último, asignaremos dentro de las propiedades de la base de datos el nuevo recurso.



Añadir nueva colección a la base de datos

Fuente: Elaboración propia

El borrado de una nueva colección se realizará como los pasos inversos realizados:

1. Borrado a través de la interfaz de recursos.
2. Borrado del disco, si se quiere borrar definitivamente.

3.4 Lenguaje de consulta para XML: XQuery (Xml Query Language)

XQuery es un lenguaje de consulta como pueda ser SQL, especializado en ficheros y datos almacenados sobre el estándar XML. Mientras que SQL es un lenguaje principalmente funcional, XQuery añade capacidades de programación. Es un estándar mantenido por el W3C, el mismo organismo que mantiene el XML, HTML y otros estándares. A partir del XQuery, tenemos el XPath, otro lenguaje de consulta sobre el XML, y que resulta ser un subconjunto del XQuery.

Algunas de las **propiedades** que nos proporciona el XQuery son:

- Extracción y manipulación de información de documentos XML.
- Expresiones similares a las usadas en SQL, expresiones FLWOR (FOR, LET, WHERE, ORDER BY y RETURN).
- Posibilidad de construir nuevos documentos a partir de una consulta.
- Basado en el modelo en árbol del propio XML.

XBase implementa tanto XPath como Xquery, tal y como la documentación oficial de BaseX nos describe. Son muchas las características y posibilidades que tenemos dentro de BaseX, desde la ejecución de sentencias FLWOR hasta sentencias mucho más complejas.

ENLACE DE INTERÉS

Accede para encontrar documentación oficial de XQuery dentro de BaseX:

3.4.1 Instrucciones basadas en Xquery

En el apartado de XQuery 3.0 y 3.1, dentro de la documentación XQuery de BaseX, nos encontramos con ejemplos y una introducción al uso de XQuery. Un ejemplo de una consulta for dentro de BaseX podría ser:

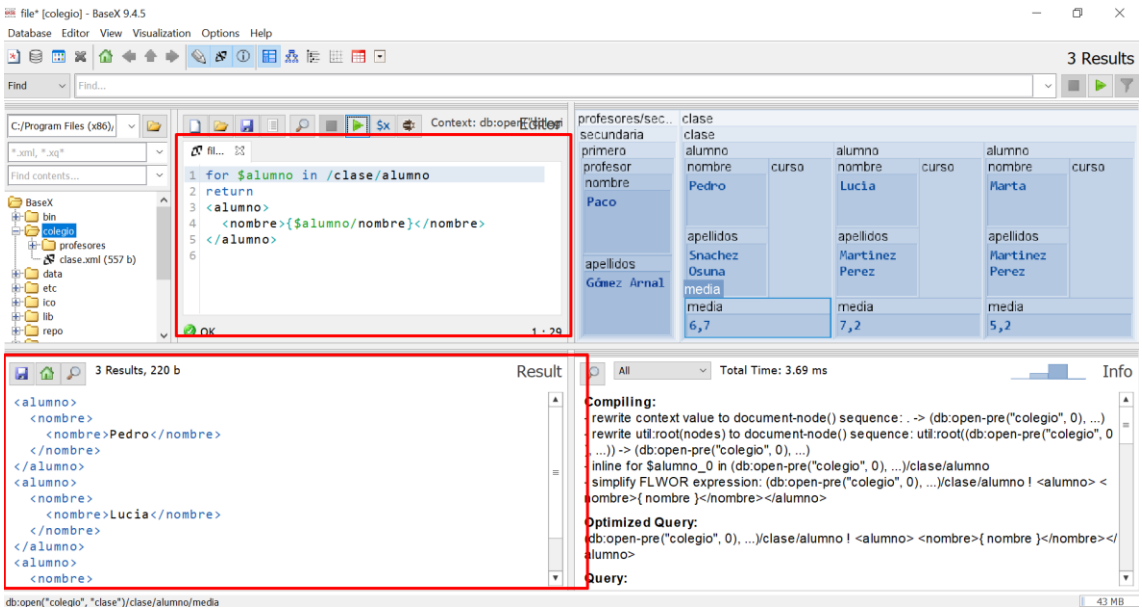
```
for $country in db:open('factbook')//country
where $country/@population > 100000000
for $city in $country//city[population > 1000000]
group by $name := $country/name[1]
count $id
return <country id='{ $id }' name='{ $name }'>{ $city/name }</country>
```

Como vemos en la anterior consulta, tenemos una forma muy potente de poder extraer y realizar búsquedas dentro de los diferentes documentos de la base de datos, donde:

- **db:open**, instrucción de XQuery que permite abrir un determinado documento dentro de la base de datos.

- **for/where**, permite realizar una búsqueda dentro del árbol de información dentro del documento.

Incluso permite extraer la información en formato XML y puede ser utilizable para la creación de nueva información a través del comando *return*. **Dentro del administrador de BaseX**, podremos realizar este tipo de consultas directamente en el file que nos aparece en la interfaz y ejecutar el comando para observar el resultado tal y como vemos en la imagen:



Ejecutar comandos XQuery

Fuente: Elaboración propia

Vemos como hemos escrito nuestro código XQuery sobre nuestra base de datos, obteniendo el resultado de la pantalla de resultados tal y como hemos marcado en el interfaz:

```
for $alumno in /clase/alumno
return
<alumno>
  <nombre>{$alumno/nombre}</nombre>
</alumno>
```

ARTÍCULO DE INTERÉS

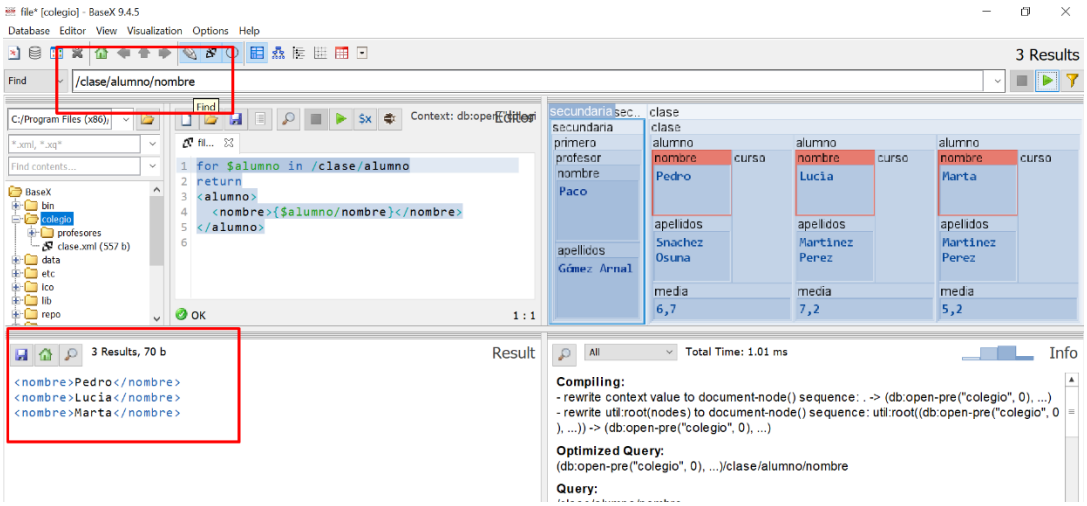
En el siguiente artículo, encontrarás un ejemplo muy completo de XQuery:

VÍDEO DE INTERÉS

Visiona este tutorial introductorio al Xquery:

3.4.2 Instrucciones basadas en XPath

Como hemos introducido anteriormente, tenemos un subconjunto de instrucciones dentro de XQuery que pueden ser de gran ayuda para realizar determinadas acciones. Con BaseX, podemos realizar esas búsquedas de forma sencilla en la barra de búsqueda tal y como podemos ver en la siguiente imagen:



XPath en BaseX

Fuente: Elaboración propia

Como podemos observar, al introducir la búsqueda /clase/alumno/nombre, el resultado será los nombres de todos los alumnos de esa base de datos. No es el objetivo estudiar el lenguaje XPath en su totalidad, pero sí el de tener la referencia de su existencia como lenguaje muy visual fácil de emplear.

3.5 Clases y métodos Java para usar con BaseX

Desde el punto de vista de desarrollador Java, es interesante contar y conocer qué librerías podemos usar para incorporar BaseX dentro de nuestras aplicaciones.

Tal y como vemos en la documentación, tenemos un conjunto de librerías para diferentes situaciones:

- Uso de BaseX en modo local.
- Uso de BaseX en modo servidor.
- Uso de BaseX en modo cliente contra servidor.
- Uso como un APIRestFul.

Nos centraremos en la primera de las modalidades, ya que nos ofrecerá la base para poder desarrollar contra el resto de funcionalidades.

ENLACE DE INTERÉS

Aquí encontrarás la documentación oficial de Java dentro de BaseX:

3.5.1 Incluir BaseX dentro de nuestros proyectos

Siguiendo la documentación de BaseX, podemos incluir dentro de nuestro proyecto BaseX simplemente añadiendo vía Maven el artefacto y la dependencia adecuada.

```
<repositories>
  <repository>
    <id>basex</id>
    <name>BaseX Maven Repository</name>
    <url>http://files.basex.org/maven</url>
  </repository>
</repositories>

…

<dependency>
  <groupId>org.basex</groupId>
  <artifactId>basex</artifactId>
  <version>7.6</version>
</dependency>
```

Si estamos usando Eclipse, podemos incorporar los plugins que nos marca la documentación para comenzar a trabajar con BaseX. Sea cual sea el mecanismo que usemos, el repositorio de basex contiene las siguientes librerías:

- Basex-core, como el principal proyecto.
- Basex-api, contiene basex apis y los servicios http.
- Basex-examples, con diferentes ejemplos.
- Basex-tests, ejemplos con junit.

PARA SABER MÁS

Aprende a incorporar BaseX en un proyecto Java en Eclipse:

3.5.2 Ejecutar comandos de forma local. Añadir, modificar o eliminar documentos y colecciones

Una de las formas de trabajar con BaseX desde nuestra aplicación es de forma local para ello debemos crear un nuevo Contexto, que nos permita ejecutar comandos para trabajar contra las bases de datos. Según la documentación del fabricante en su API, un nuevo objeto de tipo Context nos permite referenciar bases de datos abiertas, sus opciones, sesiones de cliente y otras propiedades.

En nuestra aplicación los pasos que debemos seguir serían en, primer lugar, incluir las librerías necesarias para trabajar con BaseX:

```
import org.basex.core.*;
import org.basex.core.cmd.*;
```

La base de esas librerías, tal y como hemos descrito anteriormente, es basex.xore. Una vez incluidas debemos crear un nuevo contexto que automáticamente se ligará a nuestra base de datos, bien creada por ese contexto o bien ya creada:

```
Context context = new Context();
```

A partir de ese contexto podemos ejecutar diferentes comandos dentro del paquete import *org.basex.core.cmd.** que también hemos importado. Dentro de los **comandos** podemos destacar:

CLASE	USO
Add	Añade un documento a una colección.
Close	Cierra una base de datos y libera sus recursos.
CreateDB	Crea una nueva base de datos.
DropDB	Borra una base de datos.
Execute	Ejecuta un comando.
Find	Ejecuta un comando de búsqueda Xquery.
List	Lista las posibles bases de datos.
Rename	Renombra recursos, documentos o colecciones de la base de datos.
XQuery	Evalúa el comando XQuery y lo ejecuta.

Un **ejemplo** en el que crearíamos una **base de datos**, crearíamos un documento, lo añadiríamos a esa base de datos y por último eliminaríamos la base de datos.

```
//Creamos una nueva base de datos
New CreateDB("DBExample",
"src/main/resources/xml/input.xml").execute(context);

    // Cerramos y reabrimos la base de datos
    System.out.println("\n* Close and reopen database.");

    new Close().execute(context);
    new Open("DBExample").execute(context);

    // Creamos un índice de la base de datos
    System.out.println("\n* Create a full-text index.");

    new CreateIndex("fulltext").execute(context);

    //Mostramos información de la base de datos
    System.out.println("\n* Show database information:");

    System.out.print(new InfoDB().execute(context));

    //Borramos índices
    System.out.println("\n* Drop indexes.");

    new DropIndex("text").execute(context);
    new DropIndex("attribute").execute(context);
    new DropIndex("fulltext").execute(context);

    //Borramos la base de datos
    System.out.println("\n* Drop the database.");

    new DropDB("DBExample").execute(context);
```

Para la **creación de recursos y manejo de estos**, tales como documentos o colecciones, podemos usar el comando Add, Rename o Delete:

```
new CreateDB("Collection").execute(context);
new Add("", "src/main/resources/").execute(context);
 new Optimize().execute(context);
new Rename("documento.xml", "nuevoDocumento.xml").execute(context);

new Delete("test.xml").execute(context);
```

ENLACE DE INTERÉS

Conoce más sobre JavaDoc de la API de Context dentro de BaseX:

Para realizar consultas a documentos XML con XQuery en nuestros programas en Java:

```
// Definir la consulta XQuery
String consulta = "for $alumno in /clase/alumno return
$alumno/nombre";

// Ejecutar la consulta
String resultado = new XQuery(consulta).execute(context);

// Mostrar el resultado
System.out.println(resultado);
```

EJEMPLO PRÁCTICO

Trabajas en el departamento de TI de una librería local que desea digitalizar su catálogo de libros para mejorar la gestión y accesibilidad de la información. La librería cuenta con una amplia variedad de libros categorizados por género, autor y año de publicación. Tu tarea es crear una base de datos XML nativa utilizando BaseX y desarrollar una aplicación en Java que permita al personal gestionar el catálogo.

Los pasos a seguir serían:

1. **Primero vas a proceder al diseño de la estructura del XML para el catálogo de libros.**

```
<Libro>
    <ID>LIB-0001</ID>
    <Titulo>El Ingenioso Hidalgo Don Quijote de la Mancha</Titulo>
    <Autor>Miguel de Cervantes Saavedra</Autor>
    <Genero>Ficción</Genero>
    <Precio>19.99</Precio>
    <AnoPublicacion>1605</AnoPublicacion>
</Libro>
```

2. **Ahora, añadirás la librería de BaseX al proyecto. Vas a hacerlo a través de Java Maven.**

```
<dependencies>
    <dependency>
        <groupId>org.basex</groupId>
        <artifactId>basex</artifactId>
        <version>9.7.3</version>
    </dependency>
</dependencies>
```

3. **Por último, ya solo queda implementar la aplicación principal que se encarga de la gestión de la librería.**

```
package libreriaxml;

import org.basex.core.Context;
import org.basex.core.BaseXException;
import org.basex.core.cmd.Add;
import org.basex.core.cmd.Close;
import org.basex.core.cmd.CreateDB;
import org.basex.core.cmd.Delete;
import org.basex.core.cmd.DropDB;
import org.basex.core.cmd.XQuery;

public class GestionLibreria {
    private Context context;

    public GestionLibreria() {
        context = new Context();
    }
```

```
    // Método para crear la base de datos
    public void crearBaseDatos(String nombreBD, String rutaXML) throws
BaseXException {
        new CreateDB(nombreBD, rutaXML).execute(context);
        System.out.println("Base de datos '" + nombreBD + "' creada
exitosamente.");
    }

    // Método para agregar un libro
    public void agregarLibro(String id, String titulo, String autor, String
genero, double precio, int anoPublicacion) throws BaseXException {
        String libroXML = "<Libro>" +
                          "<ID>" + id + "</ID>" +
                          "<Titulo>" + titulo + "</Titulo>" +
                          "<Autor>" + autor + "</Autor>" +
                          "<Genero>" + genero + "</Genero>" +
                          "<Precio>" + precio + "</Precio>" +
                          "<AnoPublicacion>" + anoPublicacion +
"</AnoPublicacion>" +
                          "</Libro>";
        new Add("libros/" + id + ".xml", libroXML).execute(context);
        System.out.println("Libro '" + titulo + "' agregado al catálogo.");
    }

    // Método para eliminar un libro por ID
    public void eliminarLibro(String id) throws BaseXException {
        new Delete("libros/" + id + ".xml").execute(context);
        System.out.println("Libro con ID '" + id + "' eliminado del
catálogo.");
    }

    // Método para modificar el precio de un libro
    public void modificarPrecio(String id, double nuevoPrecio) throws
BaseXException {
        String consulta = "update replace doc('libros/" + id +
".xml')/Libro/Precio with " + nuevoPrecio;
        new XQuery(consulta).execute(context);
        System.out.println("Precio del libro con ID '" + id + "' actualizado
a " + nuevoPrecio + "€.");
    }

    // Método para consultar libros por género
    public void consultarPorGenero(String genero) throws BaseXException {
        String consulta = "for $libro in //Libro[Genero = '" + genero + "']
" +
                          "return <Resultado>" +
                          "<Titulo>{$libro/Titulo/text()}</Titulo>" +
                          "<Autor>{$libro/Autor/text()}</Autor>" +
                          "</Resultado>";
        String resultado = new XQuery(consulta).execute(context);
        System.out.println("Libros del género '" + genero + "':");
        System.out.println(resultado);
    }

    // Método para cerrar la conexión
    public void cerrarConexion() throws BaseXException {
        new Close().execute(context);
        context.close();
        System.out.println("Conexión con la base de datos cerrada.");
```

```
    }

    public static void main(String[] args) {
        GestionLibreria gl = new GestionLibreria();
        try {
            // Crear la base de datos (solo la primera vez)
            // gl.crearBaseDatos("CatalogoLibreria",
"ruta/a/tu/directorio/xml");

            // Agregar libros
            gl.agregarLibro("LIB-0001", "Cien Años de Soledad", "Gabriel
García Márquez", "Ficción", 25.50, 1967);
            gl.agregarLibro("LIB-0002", "El Principito", "Antoine de Saint-
Exupéry", "Ficción", 15.00, 1943);
            gl.agregarLibro("LIB-0003", "Sapiens", "Yuval Noah Harari", "No
Ficción", 30.00, 2011);

            // Modificar el precio de un libro
            gl.modificarPrecio("LIB-0002", 17.00);

            // Consultar libros por género
            gl.consultarPorGenero("Ficción");

            // Eliminar un libro
            // gl.eliminarLibro("LIB-0003");

            // Cerrar la conexión
            gl.cerrarConexion();

        } catch (BaseXException e) {
            e.printStackTrace();
        }
    }
}
```

4. **Si lo ejecutas obtendrás el resultado:**

```
Problems  Javadoc  Declaration  Search  Console  Error Log  Coverage
<terminated> Prueba [Java Application] C:\Users\Patricio\Downloads\openjdk-21_windows-x64_bin\jdk-21\b
Libro 'Cien Años de Soledad' agregado al catálogo.
Libro 'El Principito' agregado al catálogo.
Libro 'Sapiens' agregado al catálogo.
Precio del libro con ID 'LIB-0002' actualizado a 17.0€.
Libros del género 'Ficción':
<Resultado>
  <Titulo>Cien Años de Soledad</Titulo>
  <Autor>Gabriel García Márquez</Autor>
</Resultado>
<Resultado>
  <Titulo>El Principito</Titulo>
  <Autor>Antoine de Saint-Exupéry</Autor>
</Resultado>
Conexión con la base de datos cerrada.
```

Librería BaseX

Fuente: Elaboración propia

3.5.3 Tratamiento de las excepciones

Las **excepciones** que deberemos manejar dentro de nuestras aplicaciones al trabajar con Basex de forma local se denomina BaseXException, y es una extensión de IOException, al estar trabajando con ficheros en un sistema operativo.

Esta clase manejará las excepciones provocadas por:

- Lectura de ficheros.
- Manejo de la base de datos.
- Manejo de recursos con la base de datos.

Al ser una excepción que extiende directamente de IoException, tendremos los mismo métodos y mecanismos de trabajo de esta. En el siguiente ejemplo podemos ver como un programa lo podemos envolver con esa clase.

```
public final class RunCommands {
  /**
   * Runs the example code.
   * @param args (ignored) command-line arguments
   * @throws BaseXException if a database command fails
   */
  public static void main(final String... args) throws BaseXException
{
```

PARA SABER MÁS

Aquí tienes la documentación sobre JavaDoc de la API de BaseException:

EJEMPLO PRÁCTICO

En el departamento donde trabajas, acabas de generar una nueva base de datos con BaseX y generado una colección de recursos a través del administrador visual. Sin embargo, estas mismas tareas las quieres realizar automáticamente a través de tu desarrollo hecho con Java. Has pensado en dar un ejemplo y, para ello, debes crear una base de datos de empleados, añadiendo documentos de empleados y realizando consultas XQuery para obtener los empleados.

1. **El primer paso es añadir la librería de BaseX al proyecto Java. Para ello puedes hacerlo importando el JAR a mano o a través de un nuevo proyecto Maven. Hoy vas a optar por hacerlo con Maven.**

```
<dependencies>
    <dependency>
        <groupId>org.basex</groupId>
        <artifactId>basex</artifactId>
        <version>9.7.3</version>
    </dependency>
</dependencies>
```

2. **Ahora creas tu clase Principal que operará con la base de datos XML.**

```
package empleadosxml;

import org.basex.core.Context;
import org.basex.core.BaseXException;
import org.basex.core.cmd.Add;
import org.basex.core.cmd.Close;
import org.basex.core.cmd.CreateDB;
import org.basex.core.cmd.XQuery;

public class Principal {
    public static void main(String[] args) {
        Context context = new Context();

        try {

            new CreateDB("Empleados").execute(context);
            System.out.println("Base de datos 'Empleados' creada.");

            // Agregar documentos XML a la base de datos
            new Add("empleado1.xml",
"<empleado><nombre>Ana</nombre><puesto>Desarrollador</puesto></empleado>").e
xecute(context);
            new Add("empleado2.xml",
"<empleado><nombre>Juan</nombre><puesto>Analista</puesto></empleado>").execu
te(context);
            new Add("empleado3.xml",
"<empleado><nombre>María</nombre><puesto>Gerente</puesto></empleado>").execu
te(context);
            System.out.println("Documentos agregados a la base de datos.");
```

```
            // Realizar una consulta XQuery
            String consulta = "for $e in //empleado return
<resultado><nombre>{$e/nombre/text()}</nombre><puesto>{$e/puesto/text()}</pu
esto></resultado>";
            String resultado = new XQuery(consulta).execute(context);

            // Mostrar los resultados
            System.out.println("Resultados de la consulta XQuery:");
            System.out.println(resultado);


        } catch (BaseXException e) {
            // Manejo de excepciones
            e.printStackTrace();
        } finally {
            // Cerrar la base de datos y liberar recursos
            try {
                new Close().execute(context);
            } catch (BaseXException e) {
                e.printStackTrace();
            }
            // Cerrar el contexto
            context.close();
        }
    }
}
```

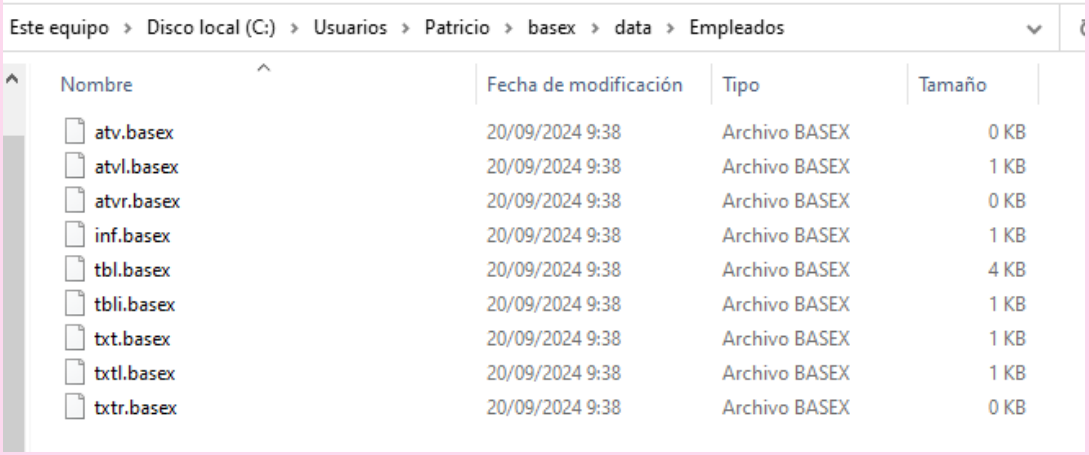
3. **Si lo ejecutas, obtienes el siguiente resultado:**

```
<terminated> Principal (7) [Java Application] C:\Users
Base de datos 'Empleados' creada.
Documentos agregados a la base de datos.
Resultados de la consulta XQuery:
<resultado>
  <nombre>Ana</nombre>
  <puesto>Desarrollador</puesto>
</resultado>
<resultado>
  <nombre>Juan</nombre>
  <puesto>Analista</puesto>
</resultado>
<resultado>
  <nombre>María</nombre>
  <puesto>Gerente</puesto>
</resultado>
```

Ejecución programa

Fuente: Elaboración propia

4. **La base de datos se almacenará en: C:\Users\[usuario]\basex\data\Empleados**



Base de datos

Fuente: Elaboración propia

RESUMEN FINAL

Las bases de datos XML y nativas XML son una solución que tuvo su auge con la estandarización y uso del XML, promocionado por el incremento de uso de las redes de ordenadores. Esta solución nos permitía almacenar información estructurada, usando un estándar que era válido no sólo para bases de datos sino para otras muchas situaciones incluidas las comunicaciones.

Con esta perspectiva hemos introducido los conceptos base sobre las bases de datos XML nativas y sus diferencias y semejanzas con las bases de datos relacionales. De esta forma, también hemos introducido dos soluciones actuales basadas en XML Nativo.

A partir de estas bases, hemos relacionado las soluciones y otras con las librerías que tenemos disponibles dentro de Java para el desarrollo de XML sobre bases de datos, la Api XQJ.

Por último, hemos trabajado con unas solución actual y concreta, proporcionándonos la visión y el detalle concreto de uso con BaseX, tanto en desde el punto de vista de administrador, como de desarrollador de aplicaciones.